```
# compute cost function helps us to compute the mean squared
error
function compute_cost(X, y, theta)
    m = size(X)[1] # number of samples

    preds = X * theta #calculate predictions using theta
    loss = preds - y #calculate error

    # Half mean squared loss
    cost =  (1/(2m)) * (loss' * loss)
    return cost

end
```

8. The cost function requires the theta value for making the predictions from which the error can be calculated. In order to update the theta value, we require the gradient descent function.

```
# Gradient Descent function to update the theta values
function gradient_descent(X, y, alpha, fit_intercept=true,
n_iter=2000)

    m = length(y) # number of training examples

    if fit_intercept
        # Add a bias
        b = ones(m, 1)
        X = hcat(b, X)
    else
        X
    end

    # Initializing theta
    theta = zeros(size(X)[2])

    # Initialise the cost vector
    cost = zeros(n_iter)

    #looping over the number of iterations
    for iter in range(1, stop=n_iter)
        pred = X * theta #predictions

        # Calculate the cost for each iter
        cost[iter] = compute_cost(X, y, theta)

        # Update the theta θ at each iter
        theta = theta - ((alpha/m) * X') * (pred - y);
    end
    return (theta, cost)
end
```

9. Finally, we are ready to train the model with our scaled and transformed data set.

```
theta, cost = gradient_descent(X_train_scaled, y_train, 0.05,
true, 1000)

# plot the cost during training
plot(cost,
     label="Cost per iter",
     ylabel="Cost",
     xlabel="Number of Iteration",
     title="Cost Per Iteration")
```

10. We will now make predictions on both the training and the testing data using our model.

```
# function used for prediction on given dataset using trained
theta values
function predict(X, theta, fit_intercept=true)
    m = size(X)[1]

    if fit_intercept
        b = ones(m)
        X = hcat(b, X)
    else
        X
    end

    predictions = X * theta

    return predictions
end


# Make predictions for both training and testing datasets
pred_train = predict(X_train_scaled, theta)
pred_test = predict(X_test_scaled, theta)

print("Predictions made on the training and testing set")
```

11. We have now reached the final step where we will verify the accuracy of our model by measuring the r-squared value of our predictions for the testing data.

```
#The function below returns the R squared value based on the
predictions (y_pred) and the ground truth values (y_true)
passed to it.

function r_squared_score(y_pred, y_true)

    # Compute sum of explained variance (SST) and sum of
```